

ITPROFEL2: OPEN SOURCE PROGRAMMING

Learning Module: Midterm - Week 4

Instructor: Henry V. Dela Cruz

I. Overview and Objectives

Welcome to Week 4. Having explored the broad landscape of open-source software, we now narrow our focus to one of the most powerful, readable, and widely adopted open-source programming languages in the world: **Python**. This week is dedicated to understanding Python's origins, the design philosophy that makes it unique, and the features that distinguish Python 3 from its predecessors.

Intended Learning Outcomes (ILOs):

- Detail the history and origin of the Python programming language.
- Identify the creator of Python and the unique reason behind the language's name.
- Trace the evolution of Python through its major version releases (1.0, 2.0, and 3.0).
- Analyze the key features of Python 3 that make it an ideal language for modern open-source development.

II. The Origins of Python

Who Invented Python?

Python was created by **Guido van Rossum**, a Dutch programmer. He began working on Python in the late 1980s at the Centrum Wiskunde & Informatica (CWI) in the Netherlands. It was initially conceived as a successor to the ABC programming language, which was capable of exception handling and interfacing with the Amoeba operating system. Van Rossum was Python's "Benevolent Dictator For Life" (BDFL) until he stepped down from the position in 2018, though he remains a deeply respected figure in the open-source community.

Why is Python called Python?

A common misconception is that Python is named after the large constrictor snake (which is why the official logo features two snake heads). However, the true origin is much more culturally rooted. When he began implementing Python, Guido van Rossum was reading the published scripts from *"Monty Python's Flying Circus,"* a famous BBC comedy series from the 1970s. He wanted a name for his new language that was short, unique, and slightly mysterious. Thus, "Python" was born.

III. The Evolution of Python

Since its inception, Python has undergone significant transformations. Its evolution highlights the open-source community's commitment to continuous improvement:

- **Python 1.0 (January 1994):** The first major release included functional programming tools like `lambda`, `map`, `filter`, and `reduce`.
- **Python 2.0 (October 2000):** This version introduced list comprehensions—a highly popular feature for manipulating lists—and a full garbage collector for automated memory management. More importantly, it marked a shift towards a more transparent and community-backed development process.
- **Python 3.0 (December 2008):** Also known as "Python 3000" or "Py3K". This was a major, backward-incompatible release. It was designed to rectify fundamental design flaws in the language. For example, the `print` statement was changed into a built-in `print()` function, and text strings were made Unicode by default.

IV. Key Features of Python 3

Python 3 is the standard for modern development. Its widespread adoption in data science, artificial intelligence, web development, and automation is due to several standout features:

- **Simple and Easy to Learn:** Python has a very simple and elegant syntax. It reads much like English, which allows developers to focus on solving the problem rather than worrying about complex language syntax.
- **Free and Open Source:** Python is developed under an OSI-approved open source license, making it freely usable and distributable, even for commercial use. It is managed by the Python Software Foundation (PSF).

- **Interpreted and Dynamically Typed:** Python code is executed line by line, which makes debugging easier. Furthermore, you do not need to declare the type of a variable when you create one; the interpreter infers it at runtime.
- **Object-Oriented and Procedural:** Python supports multiple programming paradigms. It can be written in a functional/procedural style or in a fully object-oriented style using classes and objects.
- **Extensive Standard Library:** Python follows a "batteries included" philosophy. It comes with a massive standard library that supports many common programming tasks such as connecting to web servers, reading and modifying files, and working with data.

Student Activity: The Zen of Python

Instructions for Students:

Python's design philosophy is formally documented in PEP 20 (Python Enhancement Proposal 20), written by Tim Peters. It is famously known as *The Zen of Python*.

Task 1: Using any online Python compiler (or an installed IDE), simply type the command `import this` and run the script.

Task 2: Read the output generated in the console. Choose **three** aphorisms (rules) from the list that resonate with you (for example, "Readability counts" or "Simple is better than complex").

Task 3: Write a one-page reflection on why these three specific rules are critically important when collaborating on large-scale Open Source projects with hundreds of other developers around the world.